

Clustering: K-Means, DBSCAN & Hierarchical

KAUST–Mawhiba IOAI Training

Week 3 · Day 16



جامعة الملك عبد الله
للعلوم والتقنية
King Abdullah University of
Science and Technology

KAUST Academy
King Abdullah University of Science and Technology

May 6, 2026

IOAI – Day 16

1. From Seeing Groups to Finding Groups
2. What is Clustering?
3. K-Means Clustering
4. DBSCAN: Density-Based Clustering
5. Hierarchical Clustering
6. Gaussian Mixture Models & KL Divergence
7. Comparison & Summary

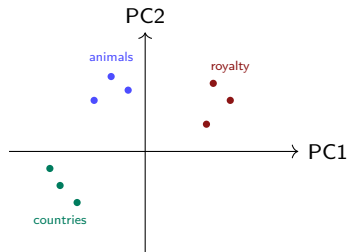
From **Seeing** Groups to **Finding** Groups

On Day 14 we learned to **project** high-dimensional data to 2D:

- ▶ **PCA:** max variance directions
- ▶ **t-SNE:** local neighborhoods
- ▶ **UMAP:** fast nonlinear reduction

When we plotted word embeddings in 2D, we saw natural groups.

But we used our eyes. **Can a computer do it automatically?**



How can a computer automatically **discover groups** in data?

Day 14 (what we did):

- ▶ Reduced dimensions
- ▶ Plotted data in 2D
- ▶ We spotted the groups

Day 16 (today):

- ▶ Give data to an **algorithm**
- ▶ It assigns each point to a group
- ▶ No human needed!

The Name for This

Automatically finding groups in data is called **clustering**. It is one of the most important tools in unsupervised learning.

We will learn **three** clustering methods, from simplest to most powerful:

1. **K-Means** — pick K centers, assign points, repeat
2. **DBSCAN** — find dense regions, ignore noise
3. **Hierarchical** — build a family tree of clusters

For each method we will:

- ▶ Walk through the idea with a **concrete example**
- ▶ See the algorithm **step by step**
- ▶ Write the **Python code**
- ▶ Discuss **when it works** and **when it fails**

What is **Clustering**?

Clustering = putting similar things into the same group.

You already do this every day:

- ▶ Sorting your playlist by mood
- ▶ Organizing your closet by type
- ▶ Grouping photos by event
- ▶ Arranging apps on your phone

Computers do it with data:

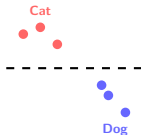
- ▶ Group customers by shopping habits
- ▶ Sort emails into spam vs. not spam
- ▶ Find types of cells in medical images
- ▶ Discover topics in news articles

Key Point

In clustering, **nobody tells us what the groups are**. The algorithm figures them out by looking at how similar the data points are to each other.

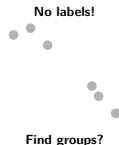
Classification (supervised)

- ▶ We *know* the labels
- ▶ Training data says: “this is a cat, this is a dog”
- ▶ The model learns to assign *new* data to known categories



Clustering (unsupervised)

- ▶ We *don't* know the labels
- ▶ Data has no tags
- ▶ The algorithm discovers groups by similarity

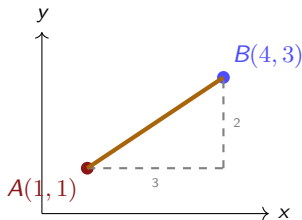


Distance: How We Measure “Similar”

To group similar points, we need a way to measure **how close** two points are.

The most common measure is **Euclidean distance** — the straight-line distance:

$$d(A, B) = \sqrt{(x_A - x_B)^2 + (y_A - y_B)^2}$$

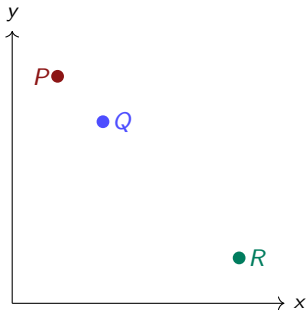


Example:

$$\begin{aligned} d(A, B) &= \sqrt{(4 - 1)^2 + (3 - 1)^2} \\ &= \sqrt{9 + 4} \\ &= \sqrt{13} \approx 3.6 \end{aligned}$$

Small distance = similar points = same cluster.

Compute the distance between each pair:



$$P = (1, 5), \quad Q = (2, 4), \quad R = (5, 1)$$

$$d(P, Q) = \sqrt{1^2 + 1^2} = \sqrt{2} \approx 1.4 \text{ (close!)}$$

$$d(P, R) = \sqrt{4^2 + 4^2} = \sqrt{32} \approx 5.7 \text{ (far!)}$$

$$d(Q, R) = \sqrt{3^2 + 3^2} = \sqrt{18} \approx 4.2$$

Which points should be in the same cluster?

P and Q are closest ($d = 1.4$), so a clustering algorithm would likely put them in the same group. R is far from both — it would go in a different cluster.

Our Running Example: 8 Students

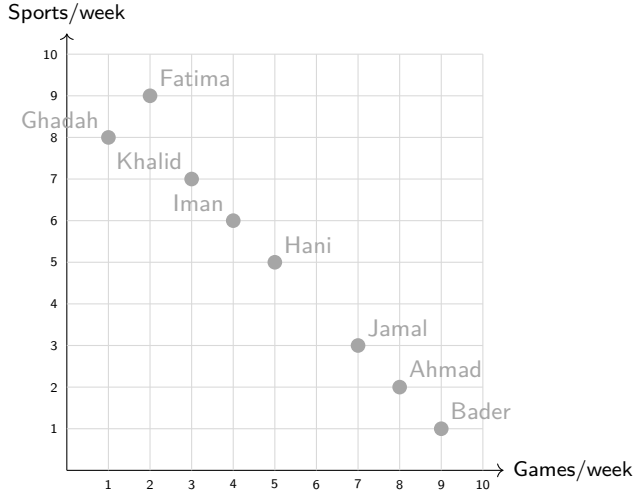
Dataset: 8 students. For each student we know two things:

- ▶ x_1 = hours of **video games** per week
- ▶ x_2 = hours of **sports** per week

Student	Games (x_1)	Sports (x_2)
Ahmad	8	2
Bader	9	1
Fatima	2	9
Ghadah	1	8
Hani	5	5
Iman	4	6
Jamal	7	3
Khalid	3	7

Question: Can we automatically find groups of students with similar habits?

Plotting Our Students



Can you see groups? Most people spot 3: gamers (bottom-right), sports fans

K-Means Clustering

K-Means is the simplest and most popular clustering algorithm.

The idea in one sentence:

Pick K center points, assign every data point to its **nearest center**, then move the centers to the **middle of their group**. Repeat until nothing changes.

- ▶ K = the number of clusters you want (you choose this!)
- ▶ **Center** = also called *centroid* = the average position of all points in a cluster
- ▶ **Nearest** = smallest Euclidean distance

1. **Initialize:** randomly pick K points as starting centers
2. **Assign:** for each data point, find the nearest center and assign it to that cluster
3. **Update:** move each center to the average of all points assigned to it

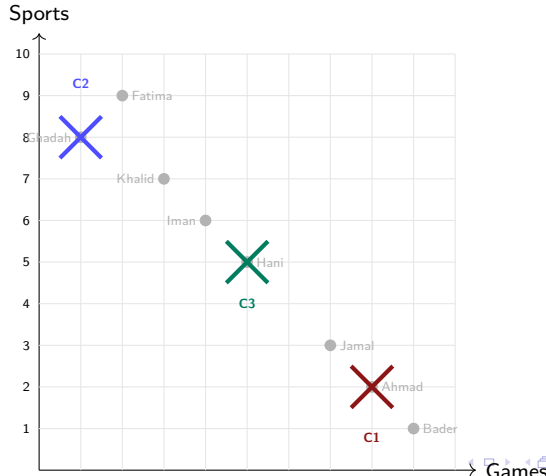
Repeat steps 2 and 3 until the assignments **stop changing**.

That's it!

The entire algorithm is just: **assign** → **move centers** → **assign** → **move centers** → ...until stable. Let's see it on our 8 students!

Step 0: Our Data + Choose $K = 3$

We have 8 students. Let's try to find $K = 3$ groups. **Pick 3 starting centers** (Ahmad, Ghadah, Hani):



Iteration 1: Assign Each Student to Nearest Center

For each student, compute the distance to **all 3 centers** and pick the closest:

Student	d to C_1	d to C_2	d to C_3	Nearest
Ahmad (8,2)	0.0	9.2	4.2	C_1
Bader (9,1)	1.4	10.6	5.7	C_1
Fatima (2,9)	9.2	1.4	5.0	C_2
Ghadah (1,8)	9.2	0.0	5.0	C_2
Hani (5,5)	4.2	5.0	0.0	C_3
Iman (4,6)	5.7	3.6	1.4	C_3
Jamal (7,3)	1.4	7.8	2.8	C_1
Khalid (3,7)	7.1	2.2	2.8	C_2

C_1 : Ahmad, Bader, Jamal C_2 : Fatima, Ghadah, Khalid C_3 : Hani, Iman

Example: distance from **Iman (4, 6)** to $C_1 = (8, 2)$

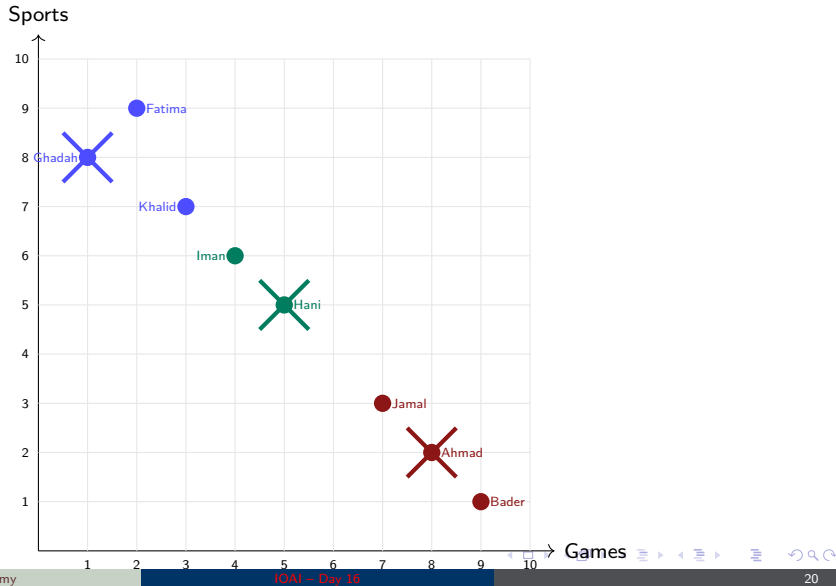
$$d = \sqrt{(4 - 8)^2 + (6 - 2)^2} = \sqrt{(-4)^2 + (4)^2} = \sqrt{16 + 16} = \sqrt{32} \approx 5.7$$

Example: distance from **Iman (4, 6)** to $C_3 = (5, 5)$

$$d = \sqrt{(4 - 5)^2 + (6 - 5)^2} = \sqrt{1 + 1} = \sqrt{2} \approx 1.4$$

Since $1.4 < 3.6 < 5.7$, Iman is closest to $C_3 \rightarrow$ assigned to the **green** cluster.

Iteration 1: Assignments Visualized



Iteration 1: Move Centers to Cluster Averages

Now **move each center** to the average position of its cluster members:

C_1 : Ahmad(8,2), Bader(9,1), Jamal(7,3)

$$C_1^{\text{new}} = \left(\frac{8 + 9 + 7}{3}, \frac{2 + 1 + 3}{3} \right) = (8.0, 2.0)$$

C_2 : Fatima(2,9), Ghadah(1,8), Khalid(3,7)

$$C_2^{\text{new}} = \left(\frac{2 + 1 + 3}{3}, \frac{9 + 8 + 7}{3} \right) = (2.0, 8.0)$$

C_3 : Hani(5,5), Iman(4,6)

$$C_3^{\text{new}} = \left(\frac{5 + 4}{2}, \frac{5 + 6}{2} \right) = (4.5, 5.5)$$

The centers barely moved! C_1 stayed at (8, 2), C_2 moved from (1, 8) to (2, 8), C_3 moved from (5, 5) to (4.5, 5.5).

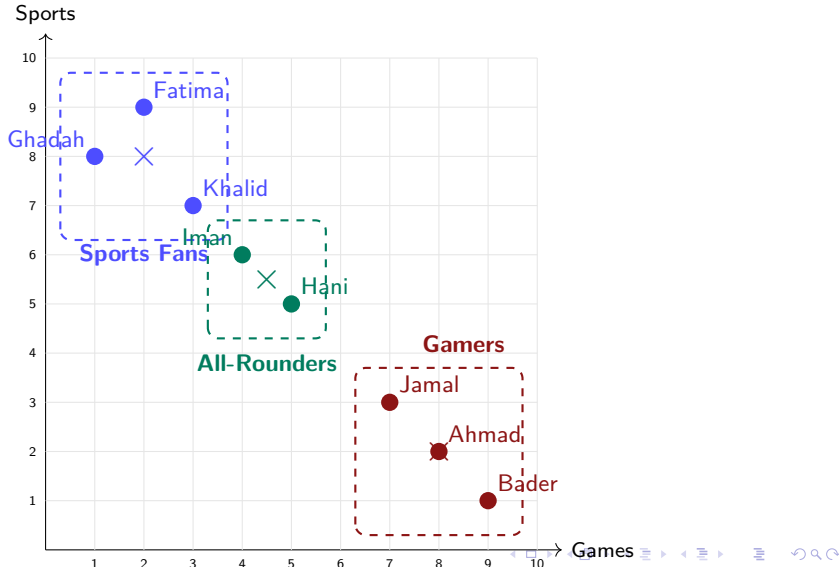
Iteration 2: Re-assign with New Centers

New centers: $C_1 = (8, 2)$, $C_2 = (2, 8)$, $C_3 = (4.5, 5.5)$

Student	d to C_1	d to C_2	d to C_3	Nearest
Ahmad (8,2)	0.0	8.5	5.0	C_1
Bader (9,1)	1.4	9.9	6.4	C_1
Fatima (2,9)	9.2	1.0	4.3	C_2
Ghadah (1,8)	9.2	1.0	4.3	C_2
Hani (5,5)	4.2	4.2	0.7	C_3
Iman (4,6)	5.7	2.8	0.7	C_3
Jamal (7,3)	1.4	7.1	3.5	C_1
Khalid (3,7)	7.1	1.4	2.1	C_2

Same assignments as before! Nothing changed $\rightarrow$ **K-Means has converged!**

Final Result: 3 Clusters Found!



Why Does K-Means Stop?

K-Means always converges (reaches a point where nothing changes).

Here is why:

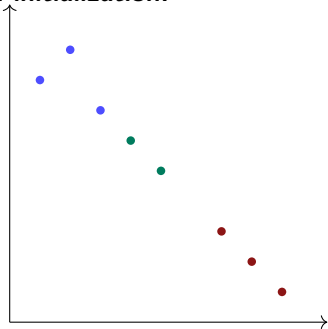
1. Each **assign step** moves points closer to their center $\rightarrow$ total distance decreases
2. Each **update step** moves centers to the middle of their group $\rightarrow$ total distance decreases
3. Total distance can only **go down or stay the same**, never go up
4. There are a finite number of possible assignments
5. So eventually the assignments **stop changing** $\rightarrow$ done!

In Practice

Watch Out: Starting Points Matter!

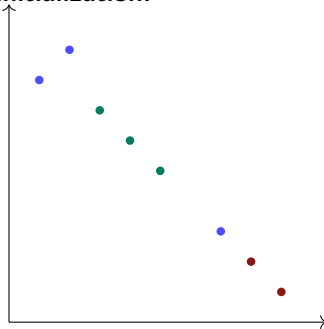
K-Means can give **different results** depending on which initial centers you pick.

Good initialization:



✓ 3 natural groups

Bad initialization:



✗ Weird split

Solution 1: run K-Means multiple times with different random starts and keep the best result. Scikit-learn does this automatically (`n_init=10`).

K-Means++ (Arthur & Vassilvitskii, 2007) picks starting centers that are **spread apart**:

1. Pick the **first center** randomly from the data points
2. For each remaining point, compute its distance d to the *nearest already-chosen center*
3. Pick the next center with probability proportional to d^2 (points **far** from existing centers are more likely to be chosen)
4. Repeat step 3 until you have K centers

Why Does This Work?

By choosing each new center *far* from the previous ones, K-Means++ avoids the “all centers start in the same cluster” problem. It gives results that are provably close to optimal.

Back to our 8 students. Suppose we pick **Ahmad (8,2)** as the first center:

Student	Position	d to Ahmad	d^2	Prob
Bader	(9,1)	1.4	2.0	3%
Fatima	(2,9)	9.2	85.0	38%
Ghadah	(1,8)	9.2	85.0	38%
Hani	(5,5)	4.2	18.0	8%
Iman	(4,6)	5.7	32.0	4%
Jamal	(7,3)	1.4	2.0	1%
Khalid	(3,7)	7.1	50.0	8%

Fatima or Ghadah (far away!) are most likely to be picked as center 2.

Then center 3 would be chosen far from *both* — likely Hani or Iman.

Result: centers start **spread across all three groups** automatically!

Great news: scikit-learn uses K-Means++ **by default!**

```
from sklearn.cluster import KMeans

# init='k-means++' is the DEFAULT
km = KMeans(n_clusters=3, init='k-means++')
labels = km.fit_predict(X)

# If you want the old random initialization:
km_random = KMeans(n_clusters=3, init='random')
```

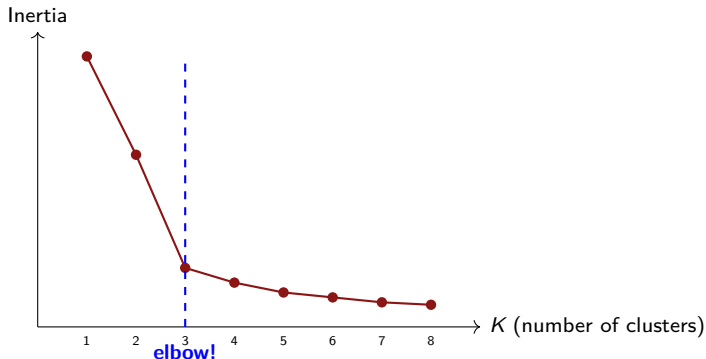
- ▶ `init='k-means++'` — smart initialization (default, use this!)
- ▶ `init='random'` — old-style random initialization
- ▶ K-Means++ is so good that scikit-learn reduced `n_init` from 10 to 1 as the default (since v1.4)

K-Means needs you to decide **how many clusters** before running.

But what if you don't know? Two popular methods:

1. **The Elbow Method:** try $K = 1, 2, 3, \dots$ and plot the total distance. Look for the “elbow” where adding more clusters stops helping much.
2. **The Silhouette Score:** measures how well each point fits in its cluster vs. the next best cluster. Higher is better (range -1 to $+1$).

Idea: run K-Means for $K = 1, 2, 3, \dots, 8$ and measure the total within-cluster distance (called *inertia*).



- From $K = 1$ to $K = 3$: inertia drops **a lot** (good improvement!)
- After $K = 3$: inertia barely changes (not worth more clusters)

For each point, the **silhouette score** measures:

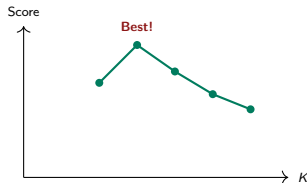
$$s(i) = \frac{b(i) - a(i)}{\max\{a(i), b(i)\}}$$

- ▶ $a(i)$ = average distance to points in the **same cluster**
- ▶ $b(i)$ = average distance to points in the **nearest other cluster**

Interpretation:

- ▶ $s \approx +1$: point is well placed
- ▶ $s \approx 0$: point is on the border
- ▶ $s \approx -1$: point is in the wrong cluster

Choose K with highest average silhouette score:



```
import numpy as np
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt

# Our 8 students: [games, sports]
X = np.array([
    [8,2], [9,1], [2,9], [1,8],
    [5,5], [4,6], [7,3], [3,7]
])
names = ['Ahmad', 'Bader', 'Fatima', 'Ghadah',
        'Hani', 'Iman', 'Jamal', 'Khalid']

# Run K-Means with K=3
kmeans = KMeans(n_clusters=3, random_state=42)
labels = kmeans.fit_predict(X)

# Print results
for name, label in zip(names, labels):
    print(f"{name} -> Cluster {label}")
```

That's it! Three lines: create, fit, predict. Scikit-learn handles all the iterations.


```
# Color each student by cluster
colors = ['red', 'blue', 'green']

for i, name in enumerate(names):
    plt.scatter(X[i,0], X[i,1],
                c=colors[labels[i]], s=100)
    plt.annotate(name, (X[i,0]+0.2, X[i,1]+0.2))

# Plot the centers
centers = kmeans.cluster_centers_
plt.scatter(centers[:,0], centers[:,1],
            c='black', marker='X', s=200)

plt.xlabel('Games per week')
plt.ylabel('Sports per week')
plt.title('K-Means Clustering (K=3)')
plt.show()
```

The `cluster_centers_` attribute gives you the final centroid positions.

```
# Try K = 1 through 8
inertias = []
K_range = range(1, 9)

for k in K_range:
    km = KMeans(n_clusters=k, random_state=42)
    km.fit(X)
    inertias.append(km.inertia_)

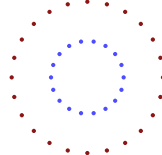
# Plot the elbow
plt.plot(K_range, inertias, 'bo-')
plt.xlabel('Number of clusters (K)')
plt.ylabel('Inertia (total within-cluster distance)')
plt.title('Elbow Method')
plt.show()
```

`km.inertia_` is the total squared distance from each point to its center. Look for the “elbow” in the plot!

- ▶ **Simple:** easy to understand and implement
- ▶ **Fast:** works on millions of data points
- ▶ **Scalable:** time grows linearly with data size
- ▶ **Always converges:** guaranteed to stop
- ▶ **Available everywhere:** built into every ML library

- ▶ **Must choose K** in advance
- ▶ **Sensitive to starting points** (can get stuck in bad solutions)
- ▶ **Assumes round clusters** (fails on weird shapes)
- ▶ **Sensitive to outliers** (one weird point pulls the center)
- ▶ **All points must belong** to a cluster (no “noise” category)

K-Means fails here:



Two rings: K-Means would split them with a straight line instead of separating inner from outer.

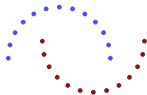
Next up: DBSCAN — a method that handles weird shapes and noise!

DBSCAN:

Density-Based Clustering

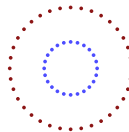
K-Means assumes clusters are **round blobs**. What about these?

Two moons:



K-Means would draw a straight line and get it wrong.

Two rings:

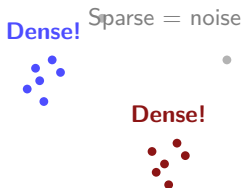


K-Means can't separate inner from outer.

We need a method that finds clusters based on **density** (how crowded an area is), not just distance to a center.

DBSCAN = *Density-Based Spatial Clustering of Applications with Noise*

The idea: a cluster is a **dense region** of points, separated from other clusters by **sparse regions**.

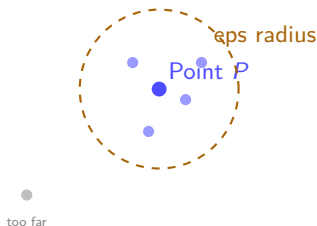


DBSCAN has **two advantages** over K-Means:

- ▶ You do **NOT** need to choose K (the number of clusters)
- ▶ It can find clusters of **any shape**, not just round ones

DBSCAN needs two numbers from you:

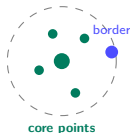
1. **eps** (ϵ): the radius of the neighborhood circle around each point. “How close must two points be to be considered neighbors?”
2. **min_samples**: the minimum number of points in a neighborhood to call it “dense.” “How many neighbors make a crowd?”



If $\text{min_samples} = 4$, then P has 4 neighbors in its eps-circle $\rightarrow P$ is a **core point**.

DBSCAN classifies every point as one of three types:

1. **Core point:** has $\geq \text{min_samples}$ neighbors within eps . These are in the *heart* of a cluster.
2. **Border point:** has $< \text{min_samples}$ neighbors, but is a neighbor of a core point. These are on the *edge* of a cluster.
3. **Noise point:** not near any core point. These are *outliers* that don't belong to any cluster.

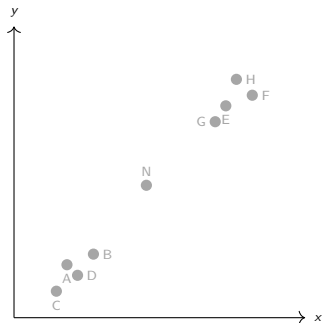


noise

Let's use a small dataset with clear dense regions and a noise point.

Settings: $\text{eps} = 1.5$, $\text{min_samples} = 3$

Point	x	y
A	1	1
B	1.5	1.2
C	0.8	0.5
D	1.2	0.8
E	4	4
F	4.5	4.2
G	3.8	3.7
H	4.2	4.5
N	2.5	2.5



For each point, count how many other points are within $\text{eps} = 1.5$:

Point	Neighbors	Count	Type
A (1, 1)	B, C, D	3	Core (≥ 3)
B (1.5, 1.2)	A, C, D	3	Core
C (0.8, 0.5)	A, B, D	3	Core
D (1.2, 0.8)	A, B, C	3	Core
E (4, 4)	F, G, H	3	Core
F (4.5, 4.2)	E, G, H	3	Core
G (3.8, 3.7)	E, F, H	3	Core
H (4.2, 4.5)	E, F, G	3	Core
N (2.5, 2.5)	—	0	Noise (< 3)

Point N is too far from any core point $\rightarrow$ it's **noise** (outlier).

Rule: two core points that are neighbors go in the same cluster.

- ▶ A is a neighbor of B, C, D $\rightarrow$ all in the same cluster
- ▶ B is a neighbor of A $\rightarrow$ already connected
- ▶ E is a neighbor of F, G, H $\rightarrow$ all in the same cluster
- ▶ No core point in group $\{A,B,C,D\}$ is a neighbor of any core point in $\{E,F,G,H\}$
- ▶ $\rightarrow$ they form **two separate clusters**



Without us telling it how many clusters!

What DBSCAN did:

1. Found core points (enough neighbors)
2. Connected neighboring core points
3. Assigned border points to nearby cores
4. Labeled lonely points as noise

Results:

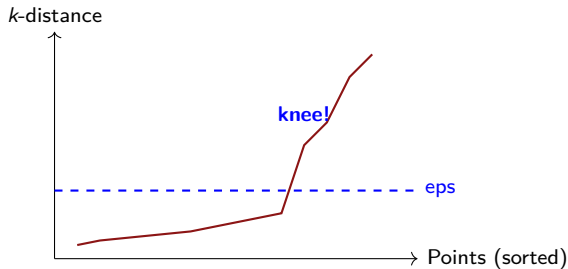
- ▶ **Cluster 1:** A, B, C, D
- ▶ **Cluster 2:** E, F, G, H
- ▶ **Noise:** N

K-Means would have forced N into a cluster. DBSCAN correctly says “N doesn’t belong anywhere.”

Choosing eps: The k-Distance Plot

How to pick a good eps? Use the **k-distance plot**:

1. For each point, compute the distance to its k -th nearest neighbor ($k = \text{min_samples}$)
2. Sort these distances from smallest to largest
3. Plot them — look for a sharp **jump** (the “knee”)



```
from sklearn.cluster import DBSCAN
import numpy as np

# 9 points: two dense groups + one noise
X = np.array([
    [1,1], [1.5,1.2], [0.8,0.5], [1.2,0.8],
    [4,4], [4.5,4.2], [3.8,3.7], [4.2,4.5],
    [2.5,2.5]      # noise point
])

db = DBSCAN(eps=1.5, min_samples=3)
labels = db.fit_predict(X)

print(labels)
# Output: [0, 0, 0, 0, 1, 1, 1, 1, -1]
# 0 = cluster 0, 1 = cluster 1, -1 = noise!
```

Key: noise points get label -1 . DBSCAN automatically found 2 clusters!

```
from sklearn.datasets import make_moons

# Generate two interleaving half-moons
X, _ = make_moons(n_samples=300, noise=0.05)

# K-Means fails:
km = KMeans(n_clusters=2).fit(X)

# DBSCAN succeeds:
db = DBSCAN(eps=0.2, min_samples=5).fit(X)

# Plot both side by side
fig, (ax1, ax2) = plt.subplots(1, 2)
ax1.scatter(X[:,0], X[:,1], c=km.labels_)
ax1.set_title('K-Means (wrong!)')

ax2.scatter(X[:,0], X[:,1], c=db.labels_)
ax2.set_title('DBSCAN (correct!)')
plt.show()
```

DBSCAN follows the **shape** of the data. K-Means draws a straight line.

- ▶ **No need to choose K :** finds the number of clusters automatically
- ▶ **Any shape:** handles moons, rings, spirals, blobs — anything!
- ▶ **Handles noise:** outliers are labeled -1 instead of forced into a cluster
- ▶ **One pass:** scans the data just once
- ▶ **Robust:** not pulled around by a single outlier

- ▶ **Sensitive to `eps` and `min_samples`:** bad settings → bad clusters
- ▶ **Struggles with varying density:** if one cluster is tight and another is spread out, one `eps` value can't handle both
- ▶ **Not great in high dimensions:** distances become less meaningful
- ▶ **Slower on big data:** without spatial indexing, it's $O(n^2)$

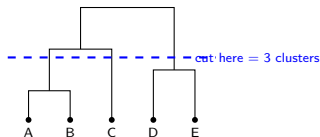
	K-Means	DBSCAN
Choose # clusters?	Yes (K)	No (automatic)
Cluster shapes	Round only	Any shape
Handles noise?	No	Yes (-1 label)
Speed	Very fast	Fast (with index)
Parameters	K	eps, min_samples
Outlier handling	Assigns to nearest	Labels as noise

Rule of Thumb

Start with **K-Means** (simple, fast). If clusters aren't round or you have outliers, switch to **DBSCAN**.

Hierarchical Clustering: Building a Family Tree of Data

The idea: start with every point as its own cluster, then **merge** the two closest clusters, one pair at a time, until everything is in one big cluster.



This tree is called a **dendrogram**. The height of each merge tells you **how different** the merged groups are.

Key advantage: you don't choose K upfront — you decide *after* by cutting the tree!

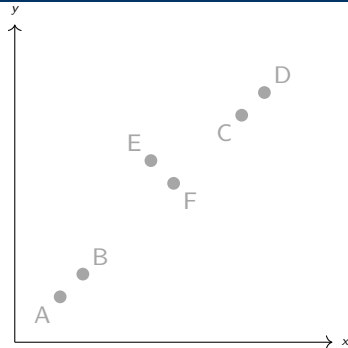
1. **Start:** every data point is its own cluster (so n clusters)
2. **Find:** the two closest clusters
3. **Merge:** combine them into one cluster
4. **Repeat** steps 2–3 until only **one cluster** remains

Why “Hierarchical”?

Each merge creates a new level in the tree. Small, tight groups merge first (bottom of the tree). Large, different groups merge last (top of the tree). This gives you a **hierarchy** — a tree of clusters inside clusters.

Running Example: 6 Points

Point	x	y
A	1	1
B	1.5	1.5
C	5	5
D	5.5	5.5
E	3	4
F	3.5	3.5

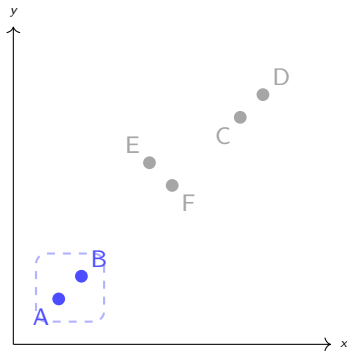


Step 1 (distances): compute distance between every pair.

- ▶ $d(A, B) = \sqrt{0.5^2 + 0.5^2} = \mathbf{0.71}$ (closest!)
- ▶ $d(C, D) = \sqrt{0.5^2 + 0.5^2} = \mathbf{0.71}$ (also closest!)
- ▶ $d(E, F) = \sqrt{0.5^2 + 0.5^2} = \mathbf{0.71}$

Merge 1: Combine the Two Closest

The closest pair is $d(A, B) = 0.71$. **Merge A and B** into cluster $\{A, B\}$.



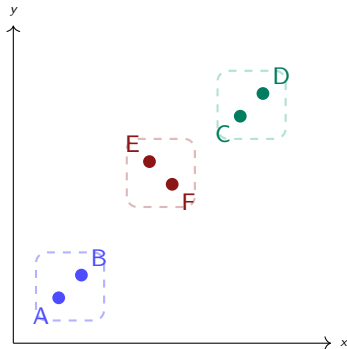
Now we have 5 clusters: $\{A, B\}$, $\{C\}$, $\{D\}$, $\{E\}$, $\{F\}$

(Ties at $d = 0.71$: we could also merge $C+D$ or $E+F$ first — order doesn't change the final tree.)

Merges 2–3: Keep Going

Merge 2: $d(C, D) = 0.71 \rightarrow$ merge into $\{C, D\}$

Merge 3: $d(E, F) = 0.71 \rightarrow$ merge into $\{E, F\}$



3 clusters remain: $\{A, B\}$, $\{C, D\}$, $\{E, F\}$

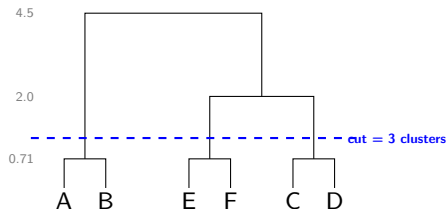
If we **stop here** we get 3 clusters. If we keep merging we get 2 then 1

Merges 4–5: All the Way to One

Merge 4: $\{E,F\}$ is closest to $\{C,D\}$ $\rightarrow$ merge into $\{C,D,E,F\}$

Merge 5: only $\{A,B\}$ and $\{C,D,E,F\}$ remain $\rightarrow$ merge into $\{A,B,C,D,E,F\}$

The complete dendrogram:



Reading the dendrogram: the *height* at which two branches merge = the distance between them. Higher merge = more different groups.

You get different numbers of clusters by cutting the dendrogram at different heights:

Cut high (e.g. height = 3):

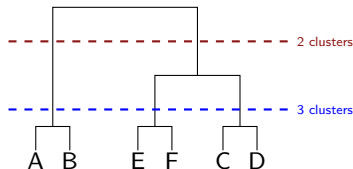
- ▶ 2 clusters: {A,B} and {C,D,E,F}

Cut medium (e.g. height = 1.5):

- ▶ 3 clusters: {A,B}, {E,F}, {C,D}

Cut low (e.g. height = 0.5):

- ▶ 6 clusters (every point alone)



Look for Big Gaps

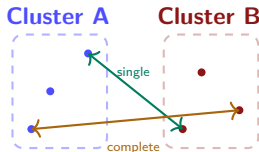
The best cut is usually where there's a **big vertical gap** in the dendrogram — that means the groups are very different from each other.

When we merge clusters, we need a distance between **groups**, not just points.

Four common methods (*linkage*):

1. **Single linkage:** distance between the *closest* pair of points → can create long, stringy clusters
2. **Complete linkage:** distance between the *farthest* pair → tends to create compact clusters
3. **Average linkage:** average distance between all pairs → balanced compromise
4. **Ward's linkage:** merges the pair that increases total variance the least → tends to create equal-sized clusters (most popular)

Default in scikit-learn: Ward's linkage. It usually works best!



- ▶ **Single:** measures the *shortest* bridge between groups
- ▶ **Complete:** measures the *longest* bridge between groups
- ▶ **Average:** takes the mean of *all* bridges
- ▶ **Ward:** minimizes variance when merging

```
from sklearn.cluster import AgglomerativeClustering
import numpy as np

X = np.array([
    [1,1],[1.5,1.5],[5,5],[5.5,5.5],[3,4],[3.5,3.5]
])

# Cut into 3 clusters
hc = AgglomerativeClustering(n_clusters=3,
                             linkage='ward')

labels = hc.fit_predict(X)
print(labels)
# Output: [0, 0, 1, 1, 2, 2]
```

You set `n_clusters` to tell it where to cut the dendrogram.

```
from scipy.cluster.hierarchy import dendrogram, linkage
import matplotlib.pyplot as plt

# Compute the linkage matrix
Z = linkage(X, method='ward')

# Plot the dendrogram
plt.figure(figsize=(8, 4))
dendrogram(Z, labels=['A','B','C','D','E','F'])
plt.title('Dendrogram (Ward linkage)')
plt.ylabel('Distance')

# Add a horizontal cut line
plt.axhline(y=2.5, color='r', linestyle='--',
            label='Cut for 3 clusters')
plt.legend()
plt.show()
```

Use `scipy.cluster.hierarchy` for the dendrogram plot. Use `sklearn` for the actual clustering.

Strengths:

- ▶ No need to choose K upfront — decide after seeing the dendrogram
- ▶ The dendrogram is a beautiful visualization
- ▶ Can reveal structure at **multiple scales**
- ▶ Different linkage methods for different needs

Weaknesses:

- ▶ **Slow:** computing all pairwise distances takes $O(n^2)$ memory and time
- ▶ **No undo:** once two clusters are merged, they can never be split
- ▶ **Sensitive** to noise and outliers
- ▶ Not practical for very large datasets (millions of points)

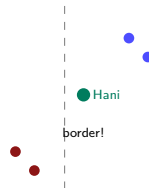
Gaussian Mixture Models (GMMs)

& KL Divergence

In K-Means, every point belongs to **exactly one** cluster. No in-between.

K-Means (hard clustering):

- ▶ Hani is in the green cluster. Period.
- ▶ Even if he's right on the border!
- ▶ No measure of *uncertainty*



What if we could say:

“Hani is **60%** likely in cluster A and **40%** likely in cluster B.”

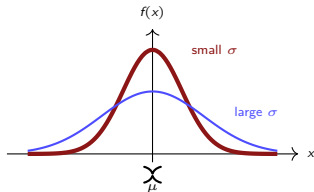
This is called *soft clustering*, and that's exactly what GMMs do!

Quick Review: The Gaussian (Normal) Distribution

A **Gaussian** (or *normal* or *bell curve*) is defined by two numbers:

- ▶ μ (**mean**): the center of the bell
- ▶ σ (**standard deviation**): how wide the bell is

$$f(x) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



In 2D

A 2D Gaussian is a “bump” or “hill” on the plane, centered at $\mu = (\mu_x, \mu_y)$ with a spread controlled by a **covariance matrix** Σ (which determines shape and tilt).

A **Gaussian Mixture Model** says:

“The data was generated by K **different Gaussians**,
each producing a different cluster.”

Each Gaussian has:

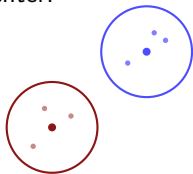
- ▶ A **mean** μ_k — the center of cluster k
- ▶ A **covariance** Σ_k — the shape and spread of cluster k
- ▶ A **weight** π_k — how big cluster k is (what fraction of points come from it)

The probability of a data point x under the mixture:

$$p(x) = \sum_{k=1}^K \pi_k \cdot \mathcal{N}(x \mid \mu_k, \Sigma_k)$$

Where $\mathcal{N}(x \mid \mu, \Sigma)$ is the Gaussian density at x with mean μ and covariance Σ .

K-Means: each cluster is a *circle* around a center.



Hard boundaries. Round shapes only.

GMM: each cluster is an *ellipse* (Gaussian) that fades out.



Soft, overlapping. Elliptical shapes.

GMMs are trained using the **Expectation-Maximization (EM)** algorithm.

It's like K-Means but with probabilities:

1. **Initialize:** pick K random Gaussian parameters (μ_k, Σ_k, π_k)
2. **E-step (Expectation):** for each point, compute the *probability* that it belongs to each Gaussian (soft assignment)

$$r_{ik} = \frac{\pi_k \cdot \mathcal{N}(x_i \mid \mu_k, \Sigma_k)}{\sum_{j=1}^K \pi_j \cdot \mathcal{N}(x_i \mid \mu_j, \Sigma_j)}$$

3. **M-step (Maximization):** update each Gaussian's μ_k, Σ_k, π_k using the weighted points (weighted by r_{ik})
4. **Repeat** until the parameters stop changing

Step	K-Means	GMM (EM)
Initialize	Pick K centers	Pick K Gaussians
Assign	Nearest center (0 or 1)	Probability (0 to 1)
Update	Move center to mean	Update μ, Σ, π
Output	Hard labels	Soft probabilities
Shape	Round	Elliptical

Key Insight

K-Means is actually a **special case** of GMMs where all covariances are equal and assignments are forced to 0 or 1 instead of probabilities.

Let's apply GMM to our 8 students with $K = 3$:
After EM converges, each student gets a **probability vector**:

	C_1	C_2	C_3
Ahmad	0.98	0.01	0.01
Bader	0.99	0.00	0.01
Fatima	0.01	0.97	0.02
Ghadah	0.00	0.98	0.02
Hani	0.05	0.10	0.85
Iman	0.02	0.25	0.73
Jamal	0.95	0.01	0.04
Khalid	0.01	0.88	0.11

Notice:

- ▶ Ahmad (0.98 for C_1) is *very sure* about his cluster
- ▶ Iman (0.73 for C_3) is *less sure* — she's between two groups
- ▶ Each row sums to 1.0
- ▶ This gives us **uncertainty** that K-Means can't provide!


```
from sklearn.mixture import GaussianMixture
import numpy as np

X = np.array([
    [8,2], [9,1], [2,9], [1,8],
    [5,5], [4,6], [7,3], [3,7]
])

# Fit a GMM with 3 components
gmm = GaussianMixture(n_components=3,
                      random_state=42)

gmm.fit(X)

# Hard labels (like K-Means)
labels = gmm.predict(X)

# Soft probabilities (the GMM advantage!)
probs = gmm.predict_proba(X)
print(probs[4]) # Hani's probabilities
# e.g. [0.05, 0.10, 0.85]
```

```
# The learned Gaussian parameters
print(gmm.means_)      # cluster centers
print(gmm.covariances_) # shapes (ellipses)
print(gmm.weights_)    # cluster sizes

# How many clusters? Use BIC or AIC
bics = []
for k in range(1, 9):
    g = GaussianMixture(n_components=k)
    g.fit(X)
    bics.append(g.bic(X))

plt.plot(range(1, 9), bics, 'bo-')
plt.xlabel('Number of components')
plt.ylabel('BIC (lower is better)')
plt.title('Choosing K for GMM')
plt.show()
```

BIC (Bayesian Information Criterion) balances fit vs. complexity. Pick the K with the **lowest BIC**.

Strengths:

- ▶ **Soft assignments:** gives probabilities, not just labels
- ▶ **Elliptical clusters:** more flexible shapes than K-Means
- ▶ **Model selection:** BIC/AIC to choose K
- ▶ **Probabilistic:** has a proper probability model behind it

Weaknesses:

- ▶ **Still needs K :** must choose number of components
- ▶ **Slower** than K-Means
- ▶ **Can overfit:** with too few points per cluster
- ▶ **Sensitive** to initialization (like K-Means)

KL Divergence: Comparing Distributions

Why Do We Need to Compare Distributions?

In GMMs, each cluster is a Gaussian distribution. Natural questions arise:

- ▶ “How **different** are cluster 1 and cluster 2?”
- ▶ “Does my model’s distribution **match** the real data?”
- ▶ “If I simplify my model, how much information do I **lose**?”

We need a way to measure the **distance between two probability distributions**.

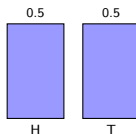
Analogy

You can measure the distance between two *points* with Euclidean distance. **KL divergence** measures the “distance” between two *distributions* (bell curves, histograms, etc.).

Imagine two coins with different bias:

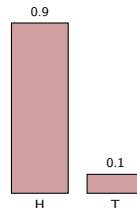
Coin P (fair):

- ▶ $P(\text{heads}) = 0.5$
- ▶ $P(\text{tails}) = 0.5$



Coin Q (biased):

- ▶ $Q(\text{heads}) = 0.9$
- ▶ $Q(\text{tails}) = 0.1$



Question: How different are P and Q ? We need a **number** to answer this.

The **Kullback-Leibler (KL) divergence** from distribution P to Q :

$$D_{\text{KL}}(P\|Q) = \sum_x P(x) \cdot \log \frac{P(x)}{Q(x)}$$

In words: for each outcome x , compare how likely it is under P vs. Q , weighted by how often it actually happens (under P).

- ▶ $D_{\text{KL}} = 0$ means P and Q are **identical**
- ▶ $D_{\text{KL}} > 0$ means P and Q are **different** (bigger = more different)
- ▶ D_{KL} is always ≥ 0 (never negative)

Compute $D_{\text{KL}}(P\|Q)$ for our two coins:

$$P = (0.5, 0.5), \quad Q = (0.9, 0.1)$$

$$\begin{aligned} D_{\text{KL}}(P\|Q) &= P(\text{H}) \cdot \log \frac{P(\text{H})}{Q(\text{H})} + P(\text{T}) \cdot \log \frac{P(\text{T})}{Q(\text{T})} \\ &= 0.5 \cdot \log \frac{0.5}{0.9} + 0.5 \cdot \log \frac{0.5}{0.1} \\ &= 0.5 \cdot \log(0.556) + 0.5 \cdot \log(5.0) \\ &= 0.5 \cdot (-0.588) + 0.5 \cdot (1.609) \\ &= -0.294 + 0.805 = \mathbf{0.511} \end{aligned}$$

(Using natural log, $\ln$.)

The two coins are moderately different ($D_{\text{KL}} = 0.511$).

Important: KL Divergence is NOT Symmetric!

$$D_{\text{KL}}(P\|Q) \neq D_{\text{KL}}(Q\|P) \quad (\text{in general})$$

Let's check: $D_{\text{KL}}(Q\|P)$ with $Q = (0.9, 0.1)$, $P = (0.5, 0.5)$:

$$\begin{aligned} D_{\text{KL}}(Q\|P) &= 0.9 \cdot \log \frac{0.9}{0.5} + 0.1 \cdot \log \frac{0.1}{0.5} \\ &= 0.9 \cdot 0.588 + 0.1 \cdot (-1.609) \\ &= 0.529 - 0.161 = \mathbf{0.368} \end{aligned}$$

$$D_{\text{KL}}(P\|Q) = 0.511 \neq 0.368 = D_{\text{KL}}(Q\|P)$$

Not a True Distance

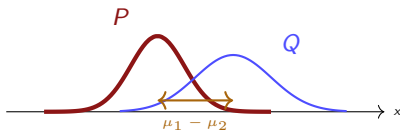
KL divergence is **not** a true distance metric because it's not symmetric. Think of it as: "how surprised would I be if I *expected* P but got Q ?" That's different from expecting Q and getting P .

For two 1D Gaussians $P = \mathcal{N}(\mu_1, \sigma_1^2)$ and $Q = \mathcal{N}(\mu_2, \sigma_2^2)$:

$$D_{\text{KL}}(P\|Q) = \log \frac{\sigma_2}{\sigma_1} + \frac{\sigma_1^2 + (\mu_1 - \mu_2)^2}{2\sigma_2^2} - \frac{1}{2}$$

What makes it large?

- ▶ The means μ_1, μ_2 are far apart (different centers)
- ▶ The variances σ_1^2, σ_2^2 are very different (different spreads)



KL divergence appears everywhere in machine learning:

1. **GMMs:** EM maximizes a quantity related to $-D_{\text{KL}}(\text{data} \parallel \text{model})$. Making KL smaller = making the model fit the data better.
2. **Variational Autoencoders (VAEs):** use KL to keep the learned distribution close to a standard Gaussian (you'll see this later!)
3. **Cross-entropy loss:** the loss function you use for classification is actually related to KL divergence:

$$\text{Cross-Entropy}(P, Q) = H(P) + D_{\text{KL}}(P \parallel Q)$$

where $H(P)$ is the entropy of the true labels.

4. **Knowledge distillation:** transferring knowledge from a big model to a small one by minimizing KL between their outputs.

```
import numpy as np
from scipy.special import kl_div
from scipy.stats import entropy

# Discrete distributions (must sum to 1)
P = np.array([0.5, 0.5])
Q = np.array([0.9, 0.1])

# Method 1: scipy.stats.entropy
kl_pq = entropy(P, Q) # KL(P || Q)
kl_qp = entropy(Q, P) # KL(Q || P)
print(f"KL(P||Q) = {kl_pq:.3f}") # 0.511
print(f"KL(Q||P) = {kl_qp:.3f}") # 0.368

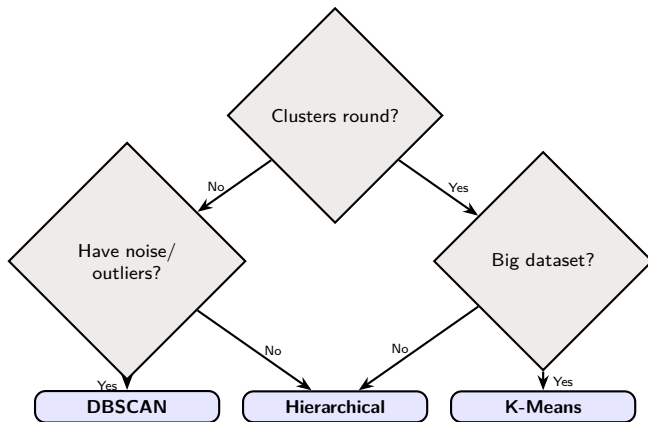
# Method 2: by hand
kl_manual = np.sum(P * np.log(P / Q))
print(f"Manual:      {kl_manual:.3f}") # 0.511
```

`scipy.stats.entropy(P, Q)` computes $D_{\text{KL}}(P||Q)$ using natural log.

1. **GMMs** model each cluster as a Gaussian distribution → *soft* cluster assignments (probabilities, not hard labels)
2. The **EM algorithm** alternates: compute probabilities (E-step), then update parameters (M-step) — just like K-Means but softer
3. **KL divergence** $D_{\text{KL}}(P\|Q)$ measures how different two distributions are — bigger means more different
4. KL is **not symmetric**: $D_{\text{KL}}(P\|Q) \neq D_{\text{KL}}(Q\|P)$
5. KL connects to **cross-entropy loss** (classification), **VAEs**, and model fitting — you'll see it again and again!

Comparison & Summary

The Big Picture: Which Method When?



	K-Means	DBSCAN	Hierarchical	GMM
Choose K ?	Yes	No	Yes (cut)	Yes
Cluster shapes	Round	Any	Any	Elliptical
Soft assign?	No	No	No	Yes
Handles noise?	No	Yes	No	No
Speed	Very fast	Fast	Slow	Moderate
Big data?	Yes	Moderate	No	Moderate

Green = advantage, **Red** = limitation

How each method handles the same tricky datasets:

Round blobs:

- ▶ K-Means: ✓ perfect
- ▶ DBSCAN: ✓ works
- ▶ Hierarchical: ✓ works

Two moons:

- ▶ K-Means: ✗ fails
- ▶ DBSCAN: ✓ great
- ▶ Hierarchical: ✓ (single linkage)

Different-sized clusters:

- ▶ K-Means: ✗ splits big cluster
- ▶ DBSCAN: ✗ one eps can't fit both
- ▶ Hierarchical: ✓ handles it

Data with noise:

- ▶ K-Means: ✗ pulls centers
- ▶ DBSCAN: ✓ labels noise
- ▶ Hierarchical: ✗ sensitive

```
from sklearn.cluster import (KMeans, DBSCAN,
                             AgglomerativeClustering)
from sklearn.mixture import GaussianMixture

# --- K-Means ---
km = KMeans(n_clusters=3)
labels_km = km.fit_predict(X)

# --- DBSCAN ---
db = DBSCAN(eps=0.5, min_samples=5)
labels_db = db.fit_predict(X)

# --- Hierarchical ---
hc = AgglomerativeClustering(n_clusters=3,
                             linkage='ward')
labels_hc = hc.fit_predict(X)

# --- GMM (soft assignments!) ---
gmm = GaussianMixture(n_components=3)
gmm.fit(X)
labels_gmm = gmm.predict(X)
probs_gmm = gmm.predict_proba(X)
```

1. **Clustering** = finding groups in data *without labels* (unsupervised)
2. **K-Means**: assign to nearest center, move center, repeat. Fast and simple, but only works for round clusters.
3. **DBSCAN**: find dense regions. Handles any shape and noise, but sensitive to ϵ .
4. **Hierarchical**: merge closest clusters bottom-up into a dendrogram. Great for exploration, but slow on big data.
5. **GMMs**: model clusters as Gaussians $\rightarrow$ soft (probabilistic) assignments. More flexible shapes than K-Means.
6. **KL divergence**: measures how different two distributions are. Connects to cross-entropy, VAEs, and model fitting.

Credits

KAUST Academy

King Abdullah University of Science and Technology

Clustering content authored for the
KAUST–Mawhiba IOAI 2026 Training Program

Week 3 · Day 16